



GAN series 1 : GAN

Yunjun Park

March 4, 2024

Motivation

- 딥러닝을 활용한 분류 모델은 relu..등의 효과로 잘 학습되며 발전
 - 생성 모델에는 몇 가지 어려움 존재
 - Intractable한 확률론적인 계산을 근사하는 것에 대한 어려움
 - Relu의 장점을 잘 활용할 수 없음
 - 이러한 문제들을 잘 회피할 수 있는 GAN
- 기존 생성 모델의 아키텍처에서 존재할 수 있는 어려움..→ GAN

Mathematical Background

- 확률분포 : 확률 변수가 특정한 값을 가질 확률을 나타내는 함수
- 확률변수를 셀 수 있는지 여부에 따라 이산/연속확률분포 나뉨
- 이미지 데이터에 대한 확률분포

이미지 데이터는 **다차원 특징 공간의 한 점**으로 표현됨

- 이미지는 많은 픽셀(파라미터)들로 구성 & 하나의 픽셀은 RGB 세 개의 채널을 가지고 있기 때문에
- 고차원 공간 상에 존재하는 분포를 학습하는 방식으로 이미지 또한 그 분포를 학습 가능(즉, 이미지의 분포를 근사하는 모델을 학습 가능)

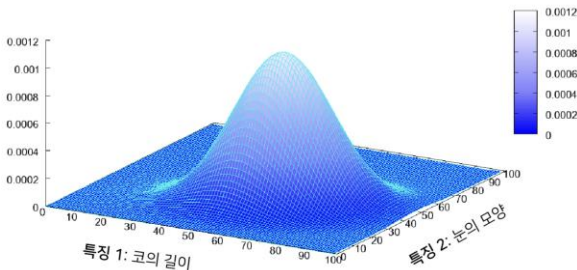
- 사람의 얼굴에는 **통계적인 평균치(눈, 코의 길이 / 목의 두께..)**가 존재할 수 있음
- 모델은 이를 수치적으로 표현할 수 있게 됨

Mathematical Background

- 이미지 데이터에 대한 확률분포

→ 이미지에서의 다양한 특징들이 각각의 확률 변수가 되는 분포를 의미

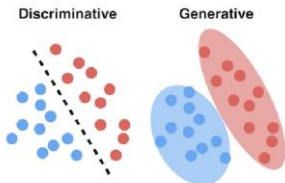
→ 이미지에 대해서 확률 분포를 학습한다고 하면 다변수 확률 분포로서 여러 개의 특징(변수)을 가지는 상황에서의 확률 분포를 학습한다고 볼 수 있음



ex) 정규분포(다변수 확률분포)

생성 모델 (Generative Models)

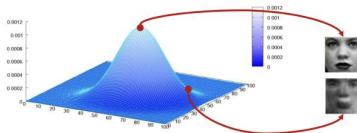
- 생성 모델은 실존하지 않지만 **있을 법한** 이미지(자연어, 오디오...가능)를 생성할 수 있는 모델을 의미



분류 vs 생성 모델

분류 : 특정한 decision boundary를 학습

생성 : 각 클래스에 대해서 적절한 분포를 학습



생성 모델 (Generative Models)

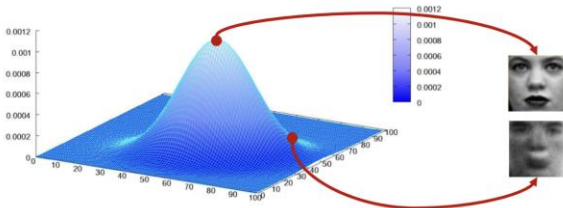
Generative Model

(produce)



An image that does not exist but is likely to exist

- A statistical model of the joint probability distribution
- An architecture to generate new data instances



→ Instance : 사진 한 장과 같은 구별되는 데이터 개체를 의미

→ 확률 분포를 잘 학습할 수 있다면 모델이 통계적인 평균치를 내재할 수 있음

→ 만약, 인간의 얼굴에 대한 분포를 잘 학습했다면..

-> 확률 값이 높은 부분에 대한 변수를 샘플링 -> 그럴싸한 이미지

-> 확률 값이 낮은 부분 -> 어색한 이미지

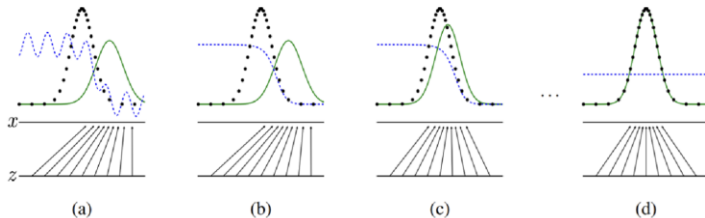
→ 분포를 잘 학습한 뒤에 확률이 높은 부분부터 출발해서 약간의 노이즈를 섞어가며 랜덤하게 샘플링을 한다면, 다양한 그럴싸한 이미지를 만들 수 있음

생성 모델 (Generative Models)의 목표

- 이미지 데이터의 분포를 근사하는 모델 G를 만드는 것이 생성 모델의 목표
- 모델 G가 잘 동작한다는 의미는 원래 이미지들의 분포를 잘 모델링할 수 있다는 것을 의미
- 2014년에 제안된 Generative Adversarial Networks (GAN)이 대표적
- GAN으로부터 매우 다양한 논문들이 파생됨

GAN 학습 과정

→ 모델 G는 원래 데이터(이미지)의 분포를 근사할 수 있도록 학습됨



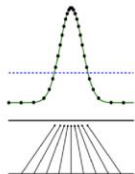
- 원본 데이터의 분포
- 생성 모델의 분포

시간이 지나면서 생성 모델 G가 원본 데이터의 분포를 학습

→ 원본 데이터가 점인 이유 : 학습할 때 가지고 있을 수 있는 데이터는 유한하기 때문

GAN 학습 과정

- 모델 G의 학습이 잘 되었다면 원본 데이터의 분포를 근사할 수 있음
- 학습이 잘 되었다면 통계적으로 평균적인 특징을 가지는 데이터를 쉽게 생성할 수 있음



있을 법한 이미지 생성



- Ex) 코, 눈의 길이는 평균적인 수치를 가지게 하되 눈썹의 두께만 더욱 진하게 하는 이미지 생성
- CGAN ; Conditional GAN

본격적인 GAN

- 생성자(generator)와 판별자(discriminator) 두 개의 네트워크를 활용한 생성 모델
- 실제 학습 후 사용하는 모델은 생성자 / 판별자는 생성자가 잘 학습하도록 도와주는 역할
- 다음의 목적 함수(objective function)를 통해 생성자는 이미지 분포를 학습할 수 있음

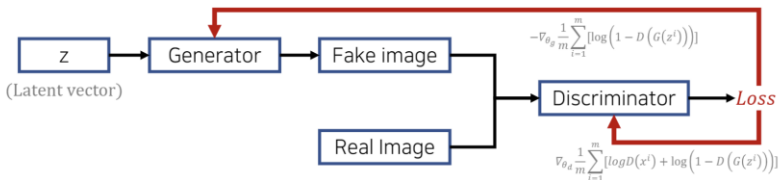
$$\min_G \max_D V(D, G) = E_{x \sim p_{data}(x)} [\log D(x)] + E_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

Generator

$G(z)$: new data instance

Discriminator

$D(x)$ = Probability: a sample came from the real distribution (Real: 1 ~ Fake: 0)

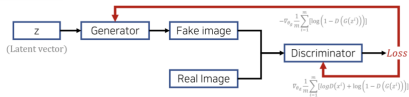


본격적인 GAN

$$\min_G \max_D V(D, G) = E_{x \sim p_{data}(x)} [\log D(x)] + E_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

Generator $G(z)$: new data instance

Discriminator $D(x)$ = Probability: a sample came from the real distribution (Real: 1 ~ Fake: 0)



→ V 는 D, G 두 개의 함수로 구성 → V 라는 함수 값을 G 는 낮추고자 하고 D 는 높이하고자 함

$$E_{x \sim p_{data}(x)} [\log D(x)]$$

→ $p_{data}(x)$, 즉 원본 데이터의 분포에서 샘플(이미지)을 하나씩 뽑아서 D 에 넣고 $\log$ 를 취한 값의 평균값(기대값) 구하기

$$E_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

→ G 는 노이즈 벡터로부터 입력을 받아서 새로운 이미지 생성

→ $p_z(z)$: 노이즈를 샘플링 할 수 있는 분포

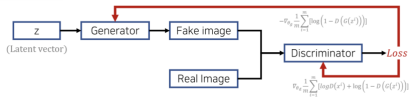
→ 노이즈를 샘플링해서 G 에 넣고 가짜 이미지 생성 후 이것을 D 에 넣은 값에 $-$ 를 붙이고 1을 더한 값에 $\log$ 취하고 이것을 평균

본격적인 GAN

$$\min_G \max_D V(D, G) = E_{x \sim p_{data}(x)} [\log D(x)] + E_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

Generator $G(z)$: new data instance

Discriminator $D(x)$ = Probability: a sample came from the real distribution (Real: 1 ~ Fake: 0)



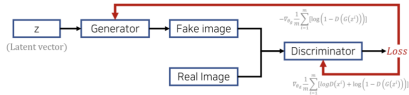
- $D(x)$ 가 0~1이므로 $\log D(x)$ 를 maximize한다는 것은 원본데이터가 1이 나오게 하도록 학습
- $\log(1-D(G(z)))$ 도 같은 원리로 G 가 생성한 가짜에 대해서는 0이 나오도록 학습
- 판별자는 학습을 함에 있어서 원본 데이터에 대해서는 1로 분류할 수 있도록 학습 & 가짜 이미지에 대해서는 0으로 분류하도록 학습
- 생성자는 목적 함수의 오른쪽 term을 minimize
- 자신이 만든 가짜 이미지가 1(진짜)이라고 인식되도록 학습
- 그래야 $\log$ 안의 값이 0이 되고 전체 term은 $-\infty$ 로 minimize 됨

본격적인 GAN

$$\min_G \max_D V(D, G) = E_{x \sim p_{data}(x)} [\log D(x)] + E_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

Generator $G(z)$: new data instance

Discriminator $D(x)$ = Probability: a sample came from the real distribution (Real: 1 ~ Fake: 0)



- G update 과정
- Noise vector(latent vector)가 들어와서 G가 Fake image 생성 → D에서 loss 구함
- 오른쪽 term을 낮춰야 하기 때문에 이 값이 줄어드는 방향으로 G를 update하기 위해 loss를 G로 미분하고 -를 붙임
- D update 과정
- Fake은 0으로 Real은 1로 부여할 수 있는 방향으로 학습
- Loss를 D로 미분 → maximize 방향이므로 gradient 앞에 + 붙임
- D, G가 동일한 식에 대해 서로 다른 목적을 가지기 때문에 일종의 minimax game 문제로 볼 수 있음
- 결국 G는 그럴싸한 이미지를 만들어 낼 수 있는 생성 모델이 될 것
- 학습 진행 : D를 먼저 학습 후 G, or 그 반대 / 매번 미니배치마다 두 개의 네트워크를 한 번씩 학습 → D, G가 각각 optimal 한 포인트로 가도록

GAN에서 기댓값 계산 방법

- 프로그램상에서 기댓값을 계산하는 가장 간단한 방법
- 단순히 모든 데이터를 하나씩 확인해서 식에 대입한 뒤에 평균을 계산하면 됨

$$E_{x \sim p_{data}(x)}[\log D(x)]$$

- 원본 데이터 분포에서의 샘플 x 를 뽑아 $\log D(x)$ 의 기댓값 계산

$$E_{z \sim p_z(z)}[\log(1 - D(G(z)))]$$

- 노이즈 분포에서의 샘플 z 를 뽑아 $\log(1 - D(G(z)))$ 의 기댓값 계산

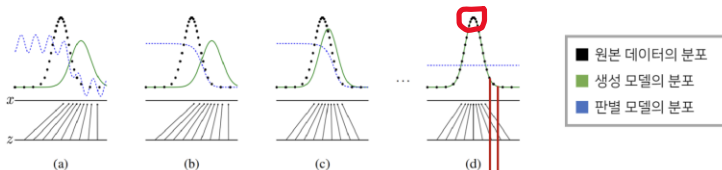
- 보통 기댓값이 붙는 거 자체가 여러 개의 데이터를 다룰 때에 대한 평균값을 나타내고자 할 때 사용
- 반복문 써서 특정 횟수만큼 반복해서 데이터를 넣고 평균 구하면 됨
- 실제 학습할 때는 미니배치 만큼 데이터를 뽑아서 넣기 때문에 ex) 배치 사이즈 32이면, 32개 만큼 데이터를 뽑아서 차례대로 함수에 넣고 기댓값(평균)을 구함

GAN의 수렴 과정

→ 공식의 목표(Goal of Formulation)

→ $P_g \rightarrow P_{data}$, $D(G(z)) \rightarrow 1/2$ (50%) ($G(z)$ is not distinguishable by D)

→ D 는 학습이 다 이루어진 뒤에는 가짜, 진짜를 더 이상 구분할 수 없기 때문에 항상 1/2



• How can the formulation lead P_g converge to P_{data} ?

key of proof

original data instance

new data instance

→ 처음에는 G 가 원본 데이터 분포를 잘 학습 못함 → D 가 잘 구별

→ 최종적으로는 $P_g \rightarrow P_{data}$, $D(G(z)) \rightarrow 1/2$ 로 수렴하는 걸 그림으로도 볼 수 있음

→ 최종 학습 후 검은색 원본 데이터 포인트 말고 다른 부분에 대해서 데이터를 꺼낸다면 이것이 새로운 이미지(new data instance)

→ 빨간 동그라미 부분처럼 확률이 높은 부분을 중심으로 약간의 noise를 섞어서 데이터 추출

→ 그럴싸한 새로운 이미지 생성 가능

증명 : Global Optimality

- p_g 가 p_{data} 로 수렴하는지를 증명
- Global Optimality : 매 상황에 대해서 G, D가 각각 어떤 포인트로 Global optimal를 가지는지에 대해 증명

Proposition: $D_G^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_g(x)}$

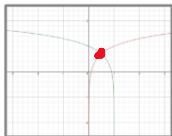
Proof: For G fixed,

$$V(G, D) = E_{x \sim p_{data}(x)}[\log D(x)] + E_{z \sim p_z(z)}[\log(1 - D(G(z)))]$$

$$= \int_x p_{data}(x) \log(D(x)) dx + \int_z p_z(z) \log(1 - D(G(z))) dz$$

$$E[X] = \int_{-\infty}^{\infty} xf(x)dx$$

Max $= \int_x p_{data}(x) \log(D(x)) + p_g(x) \log(1 - D(x)) dx$



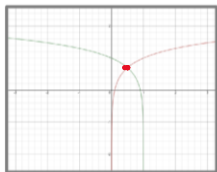
function $y \rightarrow a \log(y) + b \log(1 - y)$ achieves its maximum in $[0, 1]$ at $\frac{a}{a + b}$

same as optimal control: $\frac{\delta V(G, D)}{\delta D} [D^*(x)] = 0$

- 증명 line 2->3 부분 : z 도메인에서 샘플링 된 노이즈 벡터를 G에 넣어서 데이터 x 를 만들어 낼 수 있기 때문에 이 과정을 $z \rightarrow x$ 로 매핑하는 걸로 볼 수 있음

- $G(z) \rightarrow x, dz \rightarrow dx$: x 로 치환해서 하나의 적분식으로 표현할 수 있음

증명 : Global Optimality



function $y \rightarrow a \log(y) + b \log(1 - y)$ achieves its maximum in $[0, 1]$ at $\frac{a}{a + b}$

same as *optimal control*: $\frac{\delta V(G, D)}{\delta D} [D^*(x)] = 0$

- 빨간색 포인트가 가장 큰 값을 가지는 위치임을 직관적으로 알 수 있음
- 밑줄 친 식에 대해서 y 로 미분해서 그 도함수가 0을 가질 때를 계산해보면 바로 알 수 있음
- a, b 가 상수가 아니라 p_{data}, p_g 처럼 분포를 가질 때에도 마찬가지로 적용 가능

증명 : Global Optimality

→ 생성자의 Global optimal point

Proposition: Global optimum point is $p_g = p_{data}$

Proof:

$$C(G) = \max_D V(G, D) = E_{x \sim p_{data}(x)} [\log D^*(x)] + E_{z \sim p_g(z)} [\log(1 - D^*(G(z)))]$$

$$D_G^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_g(x)}$$

$$= E_{x \sim p_{data}(x)} \left[\log \frac{p_{data}(x)}{p_{data}(x) + p_g(x)} \right] + E_{x \sim p_g(x)} \left[\log \frac{p_g(x)}{p_{data}(x) + p_g(x)} \right]$$

removed
when $p_g = p_{data}$

$$= E_{x \sim p_{data}(x)} \left[\log \frac{2 * p_{data}(x)}{p_{data}(x) + p_g(x)} \right] + E_{x \sim p_g(x)} \left[\log \frac{2 * p_g(x)}{p_{data}(x) + p_g(x)} \right] - \log(4)$$

$$KL(p_{data} || p_g) = \int_{-\infty}^{\infty} p_{data}(x) \log \left(\frac{p_{data}(x)}{p_g(x)} \right) dx$$

$$= KL(p_{data} || \frac{p_{data}(x) + p_g(x)}{2}) + KL(p_g || \frac{p_{data}(x) + p_g(x)}{2}) - \log(4)$$

$$= 2 * JSD(p_{data} || p_g) - \log(4)$$

$$JSD(p || q) = \frac{1}{2} KL(p || \frac{p+q}{2}) + \frac{1}{2} KL(q || \frac{p+q}{2})$$

증명 : Global Optimality

Proposition: Global optimum point is $p_g = p_{data}$

Proof:

$$\begin{aligned}
 C(G) &= \max_D V(G, D) = E_{x \sim p_{data}(x)} [\log D^*(x)] + E_{x \sim p_g(x)} [\log(1 - D^*(G(x)))] \\
 &= E_{x \sim p_{data}(x)} \left[\log \frac{p_{data}(x)}{p_{data}(x) + p_g(x)} \right] + E_{x \sim p_g(x)} \left[\log \frac{p_g(x)}{p_{data}(x) + p_g(x)} \right] \\
 &\quad \left(D_G^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_g(x)} \right) \\
 &= E_{x \sim p_{data}(x)} \left[\log \frac{2 * p_{data}(x)}{p_{data}(x) + p_g(x)} \right] + E_{x \sim p_g(x)} \left[\log \frac{2 * p_g(x)}{p_{data}(x) + p_g(x)} \right] - \log(4) \\
 &\quad \left(\text{removed where } p_g = p_{data} \right) \\
 &= KL(p_{data} || p_g) - \log(4) \\
 &\quad \left(KL(p_{data} || p_g) = \int_{-\infty}^{\infty} p_{data}(x) \log \left(\frac{p_{data}(x)}{p_g(x)} \right) dx \right) \\
 &= 2 * JSD(p_{data} || p_g) - \log(4) \\
 &\quad \left(JSD(p || q) = \frac{1}{2} KL(p || \frac{p+q}{2}) + \frac{1}{2} KL(q || \frac{p+q}{2}) \right)
 \end{aligned}$$

- 별도의 C함수 정의 -> 특정한 fixed G에 대해 global optimal을 가지는 D에 대한 V함수
- Line 2 : log안에 2 곱하고 뒤에 -log(4) -> 증명의 편의성 때문
- 빨간 네모 부분 -> KL Divergence(두 분포가 얼마나 차이가 나는지(수치적으로))로 치환
- 이것도 증명의 편의성 때문 -> Jensen-Shannon divergence (JSD)를 이용하려고
- KL Divergence는 일반적으로 distance metric으로 활용이 어려움
- JSD는 두 분포의 distance를 구하는데 사용
- Distance metric이므로 최소값이 0 -> 즉, $p_g = p_{data}$ 일때 0이 됨
- 결국 최소값으로 -log(4)를 얻음
- 이런 global optimal point를 얻게 해주는 유일한 sol은 $p_g = p_{data}$ 일때
- 생성자는 D가 이미 잘 수렴해서 global optimal을 가지고 있다고 가정한 상태에서 생성자가 잘 학습된다면 -log(4)를 가질 수 있도록 잘 수렴해서 p_{data} 와 같은 분포의 형태로 수렴
- D, G에 대해 Global optimal이 존재할 수 있다는 걸 증명 but 학습이 잘 되어서 global optima에 잘 도달할 수 있는가는 다른 문제 -> GAN은 학습이 어렵다고 함 -> 나중에 다른 논문들이 개선

증명 : Global Optimality

for the number of training iterations do

for k steps do

Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.

Sample minibatch of m examples $\{x^{(1)}, \dots, x^{(m)}\}$ from data generating distribution $p_{data}(x)$.

Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m [\log D(x^i) + \log (1 - D(G(z^i)))].$$

end for

Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.

Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log (1 - D(G(z^i))).$$

end for

The gradient-based updates can use any standard gradient-based learning rule. They used momentum.

→ 매 epoch 당 k번 D를 학습한 뒤 G를 학습

증명 : Global Optimality



- Not cherry-picked
- **Not memorized** the training set
- Competitive with the better generative models
- Images represent sharp



Fig. Visualization of samples from the model



Fig. Digits obtained by linearly interpolating between coordinates in z space of the model

- 만든 이미지는 별도 생성이 아니고 랜덤하게 만든 것
- 학습데이터 단순 암기 아님
- 마지막 Column 데이터 : 바로 왼쪽 데이터와 최대한 가까운, nearest neighbor에 해당하는 학습 데이터를 뽑은 것 -> 만들어진 이미지와 엄밀한 차이 존재 -> 있을 법한 이미지를 잘 생성함

Reference

<https://www.youtube.com/watch?v=AVvIDmhHgC4>

https://github.com/ndb796/Deep-Learning-Paper-Review-and-Practice/blob/master/code_practices/GAN_for_MNIST_Tutorial.ipynb

https://github.com/rickiepark/Generative_Deep_Learning_2nd_Edition/blob/main/notebooks/04_gan/01_dcgan/dcgan.ipynb

만들면서 배우는 생성 AI



GAN series 2 :
DCGAN & WGAN-GP & **CGAN**

Yunjun Park

March 11, 2024

Motivation

- 일반적인 GAN은 생성하려는 이미지의 유형(예: 남성이나 여성 얼굴, 크거나 작은 벽돌)을 제어할 수 없음
- 잠재 공간에서 랜덤한 한 포인트를 샘플링할 수는 있지만, 어떤 잠재 변수를 선택하면 어떤 종류의 이미지가 생성될지 쉽게 파악할 수 없음
 - 출력을 제어할 수 있는 GAN, 즉 Conditional GAN(CGAN)
- Conditional GAN의 분명한 목적
 - Condition이라는 조건을 나타내는 변수를 추가함으로써 데이터 생성을 '내 입맛대로' 할 수 있도록 한 것!

Motivation

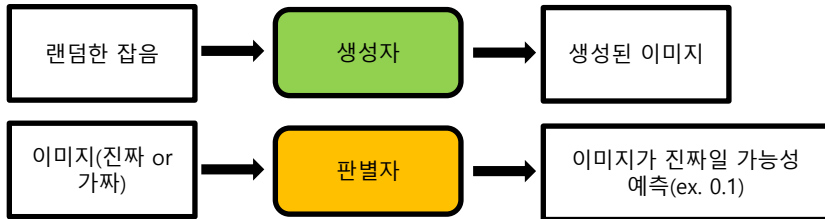
- Role of original GAN from the generative modeling perspective
 - An implicit method of generating x by $G(z)$, such as $p(x|z)$
 - Original GAN has a pure noise modeling on z , i.e. $N(0,1)$ or uniform distribution
 - **Therefore, modeling on a conditional probability is needed**
- Variation of Probabilistic model
 - GAN = NN + **Probabilistic model** ; $P(z)$, $P(x)$ 만 다루었음
 - 이 부분에 처음 variation을 줄 수 있는 것이 Conditional Probability
 - $P(z|y)$, $P(x|y)$

표준 GAN vs CGAN

- 주요 차이점
- CGAN에서는 레이블과 관련된 추가 정보(y)를 생성자와 비평가에 전달한다는 점
- Generator : 이 정보를 원한 인코딩된 벡터로 잠재 공간 샘플에 단순히 추가
Discriminator : 레이블 정보를 RGB 이미지에 추가 채널로 추가
→ 입력 이미지와 동일한 크기가 될 때까지 원한 인코딩된 벡터를 반복

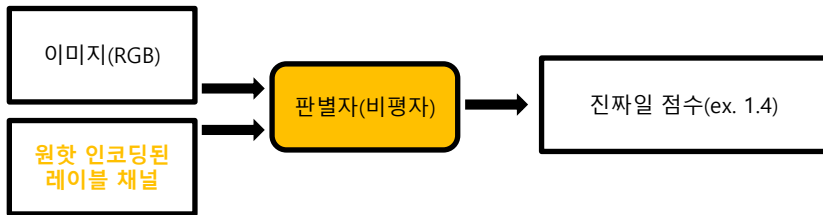
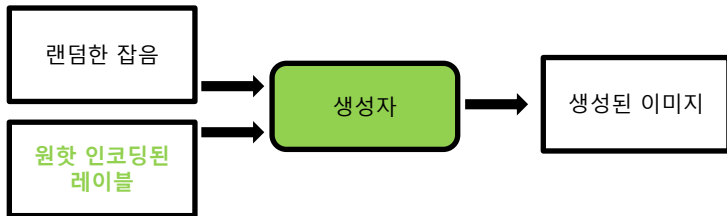
표준 GAN vs CGAN

- GAN에서 두 네트워크의 입력 및 출력



표준 GAN vs CGAN

- CGAN에서 두 네트워크의 입력 및 출력



본격적인 CGAN

- Generation 되는 과정에 있어서 condition을 걸고 싶음
Ex) X: 28x28의 MNIST image, y: 0-9의 class(categorical) label
→ y=0이면 숫자 0의 이미지를 생성!
- 그럼 먼저 Generator가 어떤 condition 하에 생성하는지를 알아야 함
: $G(z)=x \rightarrow G(z|y)=x$;
→ y가 given인 상태에서 z입력이 오면 x를 만들어 낸다!
→ Given y 이긴 하지만 결국 z, y를 모두 입력 받는 형태가 될 것
- $G(z) = x \rightarrow G(z|y) = x \rightarrow NN_G(z, y; w_G) = x$ [NN : Neural net, input : z, y]
- The generator takes the condition as an additional input
- $D(x) = p \rightarrow D(x|y) = p \rightarrow NN_D(x, y; w_D) = p$ [NN : Neural net, input : x, y]
- The discriminator takes the condition as an additional input
- y라는 조건 하에서 x가 생성 되었다고 discriminator에게도 알려줄 수 있음
(generating 할 때도 y를 조건으로 주었으니까)

본격적인 CGAN

- Generator에서 입력 노이즈인 $p_z(z)$ 와 y 는 joint hidden representation으로 결합되며, D 와의 적대적 훈련 프레임워크는 이러한 숨겨진 representation을 구성하는 방법에 상당한 유연성을 제공

Discriminator에서는 x 와 y 는 입력과 discriminative function으로 표시

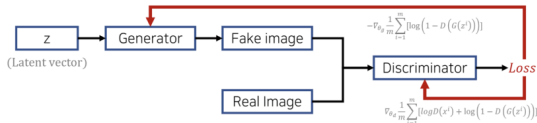
- $$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x}|\mathbf{y})] + \mathbb{E}_{\mathbf{z} \sim p_z(\mathbf{z})} [\log(1 - D(G(\mathbf{z}|\mathbf{y})))] \quad (2)$$

<CGAN>

$$\min_G \max_D V(D, G) = E_{x \sim p_{\text{data}}(x)} [\log D(x)] + E_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

- Generator $G(z)$: new data instance

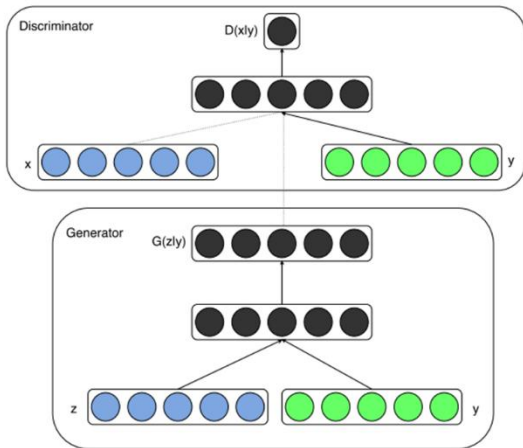
Discriminator $D(x)$ = Probability: a sample came from the real distribution (Real: 1 ~ Fake: 0)



목적함수에 단순히 y (조건)만 추가
→
조건부 확률로 표시

<GAN>

본격적인 CGAN



- 기존의 GAN 모델에서 단 하나, y 라는 **condition** 값이 추가됨.

→

이 y 값은 Generator와 Discriminator의 Input값에 들어갈 때, 단순히 이어붙여주면 된다.
(매우 간단)

Figure 1: Conditional adversarial net

본격적인 CGAN

- Ex) MNIST 데이터셋을 GAN모델로 학습하고 생성한다고 생각
 - 어느정도 Generator 가 생성되었을 때, 하나의 데이터를 샘플링해보면 그 데이터 샘플은 분명 그럴듯한 숫자 모양을 하고 있을 것
 - 하지만, 그 숫자가 무슨 label에 해당하는 숫자인지는 까보기 전까지 구현한 사람도, 샘플링한 사람도 알지 못함
 - 하지만 Conditional GAN을 통해 이러한 문제를 해결할 수 있음
 - 학습 과정에서 숫자 0에 해당하는 데이터를 학습시킬때 latent 벡터 z 옆에 **condition 변수** `[1, 0, 0, 0, 0, 0, 0, 0, 0]` 를 이어붙여줘보자. (MNIST는 숫자이므로 총 10개의 label이 있고 이를 one-hot encoding 하여 벡터로 나타냄)
 - 또한, Generator를 통해 학습시킨 값 $G(z)$ 를 Discriminator에 넣어줄때도 동일하게 **condition 변수** `[1, 0, ..., 0]` 을 이어붙여준채로 학습을 시켜보자.

본격적인 CGAN

- 마찬가지로 만약 2라는 데이터를 학습시킬때는 z 벡터와 $G(z)$ 값 뒤에 **condition 변수** $[0, 0, 1, 0, 0, 0, 0, 0, 0]$ 를 붙여주면 됨
- 이렇게 학습시킬 경우, 나중에 학습 과정이 끝나고 샘플링 할때 $G(z)$ 에서 z 뒤에 자신이 원하는 label에 해당하는 **condition 변수**를 이어붙여준다면, 자신이 원하는 label에 해당하는 샘플을 얻어낼 수 있음
- 즉 한마디로 정리해보면, GAN 학습과정에서 y 라는 **condition 변수**를 추가함으로써 자신이 원하는 label의 데이터를 생산해낼 수 있도록 만든 GAN 모델. 이라고 보면 된다.
- 이렇게 단순해서인지 Conditional GAN의 loss 함수(Objective function)도 그냥 GAN 함수에 비해 단지 y 를 조건부 확률로 추가해준 것 밖에는 없다.

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x}|\mathbf{y})] + \mathbb{E}_{\mathbf{z} \sim p_z(\mathbf{z})} [\log(1 - D(G(\mathbf{z}|\mathbf{y})))]$$

- 이외에도 논문을 보면, image 데이터를 CNN 연산하여 나온 fc layer를 condition으로 주고, 해당 사진을 설명하는 tag를 생성해낼 수 있는 모델도 확인해볼 수 있으니, 직접 논문을 읽어보는 것을 권장한다.

CGAN 분석

- y라고 하는 것에는 다양한 것들이 들어갈 수 있음!

1. Semantic meaning을 지닌 숫자
2. style 정보
3. 만화체
4. Image edge

→ 이러한 condition을 가정한 generation이 일어날 것!

CGAN 분석

- 특정 원핫 인코딩된 레이블을 생성자의 입력에 전달하여 CGAN 출력을 제어할 수 있음
 - ex) 금발이 아닌 얼굴을 생성하려면 벡터 $[1,0]$ 을 전달. 금발 머리의 얼굴을 생성하려면 벡터 $[0,1]$ 을 전달
- CGAN이 레이블 벡터를 사용하여 이미지의 머리 색깔 속성만 제어하는 방법을 학습했음을 알 수 있음
 - 이미지의 나머지 부분은 거의 변하지 않는다는 점이 인상적
 - 이는 GAN이 개별 특성이 서로 분리되도록 잠재 공간의 포인트를 구성할 수 있다는 증거
- 데이터셋에 레이블이 있는 경우
 - 레이블로 생성된 출력에 조건을 부여할 필요가 없더라도 GAN의 입력에 레이블을 포함하는 것이 좋음. 이렇게 하면 일반적으로 생성된 이미지의 품질을 높이는 경향이 있기 때문.
 - 레이블을 픽셀 입력에 추가할 수 있는 매우 유용한 정보라고 생각하면 됨

CGAN 분석

● CGAN_test_result_even_num



실제 코드를 돌려 GAN을 학습한 뒤, check_condition을 통해 확인한 결과

실제로 내가 원하는 condition대로 잘 학습된 것을 확인할 수 있다.

● CGAN_test_result_0_9



만약 기본 GAN을 통해 학습시켰다면, 생성된 이미지(그림)에 무작위의 숫자들이 들어갔을 것이다.

Reference

<https://www.youtube.com/watch?v=ZVdbELciF2o>

<https://ddongwon.tistory.com/126>

<https://github.com/godeastone/GAN-torch/blob/main/models/Conditional%20GAN.py>